

Chapitre 2 : Horloge logique

Horloge : pas possible d'horloge global, mais horloge algorithmique

On veut avoir une méthode algorithmique pour pouvoir "dater" des événements qui existent dans 1 système distribué.

Les date permettent de lier causalement les événements d'un système distribué

① Horloge de Lamport.

$$\Pi = P_1 P_2 \dots P_n$$

↑
système distribué

↑
processeur (noeud)

$P_i : e_i^1 e_i^2 \dots e_i^k$: séquence ordonné d'événement chronologique

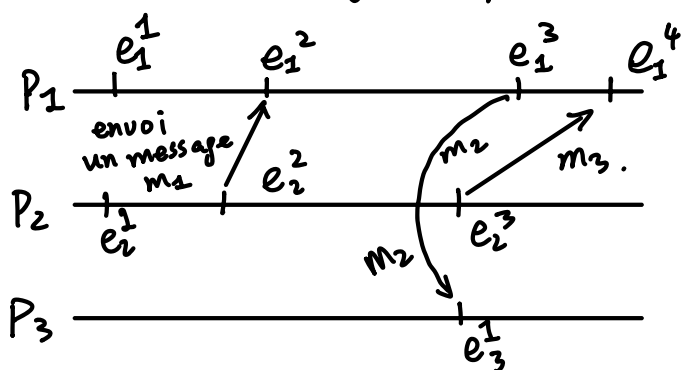
↑
événement sur le processeur i

e_i^1, e_i^k : événements locaux au processeur et on $e_i^1 < e_i^k$ il arrive avant

3 type événement :

- réception de message
- envoi de message
- calcul interne

Représentation graphique :



$e_1^1 \rightarrow e_1^4$ Local
 $e_2^2 \rightarrow e_2^3$ message.
 $e_2^2 \rightarrow e_1^3$ Transitivité

e_1^1 : événement interne

e_2^2 : envoi de message

e_1^3 : réception de message

e précède causalement e' (on écrit $e \rightarrow e'$)

ssi :

- e et e' locaux à 1 même site et e apparaît avant e'
- Soit e' correspond à réception d'un message (MESS) envoyé par e
- il existe e'' tq

$e \rightarrow e''$ et $e'' \rightarrow e'$ (transitivité)

fonction :

à chaque événement e : $H(e)$ valeur d'horloge

↓
couple

(h_i, i)

↑
estampille ↑
identifiant du site

Comment construire les estampilles :

- Init : $h_i = 0$

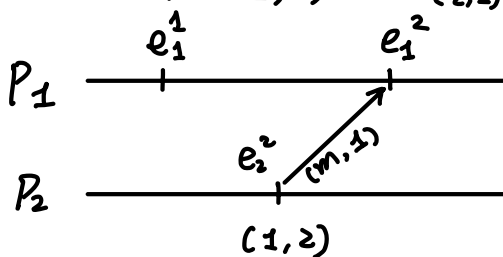
- événement local : $h_i \leftarrow h_i + 1$
 $H(e) \leftarrow (h_i, i)$

- Émission message : $h_i \leftarrow h_i + 1$
 $H(e) \leftarrow (h_i, i)$

et envoie de (m, h_i)

- Réception message (m, h) : $h_i \leftarrow \max(h_i, h) + 1$
 $H(e) \leftarrow (h_i, i)$

$H(e_1^1) = (1, 1)$ $(2, 1) = H(e_1^2)$



On peut "comparer" des valeurs d'horloge et ordonner les événements

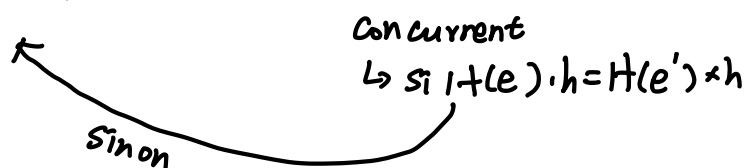
$e < e'$ ssi $\left\{ \begin{array}{l} H(e) \times h < H(e') \times h \\ \quad \uparrow \text{valeur d'estampille} \\ \text{ou} \\ H(e) \times h = H(e') \times h \text{ et } H(e) \times id < H(e') \times id \end{array} \right.$

Propriété :

$e \rightarrow e'$ alors $H(e) < H(e')$

par contre, $H(e) < H(e') \nRightarrow e \rightarrow e'$

seulement que $e \rightarrow e'$ ou $e \parallel e'$



grâce à cette horloge, on peut construire un ordre total $\ll$ sur les événements

$e_1 \ll e_2 \ll \dots \ll e_k$ qui ordonne les événements.

↳ construction : ordonner les événements par valeur d'horloge croissante.

$$e_1 \ll e_2 \Rightarrow (e_1 \rightarrow e_2) \vee (e_1 \parallel e_2)$$

Horloge vectorielle de Mattern :

Propriété $V(e) < V(e') \Leftrightarrow e \rightarrow e'$

message de taille $O(n)$

Construction d'horloge.

Initialement : chaque site contient 1 vecteur de taille n la taille du système, initialisé à 0 : le tableau est indexé par les noms des sites.

- à chaque événement sur i : $V_i[i]++$

- lors d'une émission de message, le vecteur est envoyé avec le message.

- lors d'une réception d'un vecteur D : $\forall j \neq i, \forall i[j] \leftarrow \max(V_i[j], D_i[j])$

Soit $V = V(e)$, $V'(e) = V(e')$

alors $V \leq V'$ ssi : $V[j] \leq V'[j] \forall j \in [m]$

$V < V'$ ssi $\exists k$ tq $V[k] < V'[k]$ et $\forall j \neq k, V[j] \leq V'[j]$

incomparable si $\neg(V < V') \wedge \neg(V \leq V')$

l'exclusion mutuelle :

plusieurs sites d'un réseau veulent accéder à une section critique (SC)

Un algo qui résout ce problème doit vérifier :

- un site qui demande la SC y accède en temps fini
- un seul noeud peut accéder à la SC à la fois

2 type d'algo :

- basé sur des demandes : 1 noeud qui veut la SC va envoyer des messages de demande.

hypothèse : graphe complet

- basé sur la circulation d'un jeton "automatiquement".

propriété (II) vérifiée par contre attention à la propriété (I)